01 — قاعدة البيانات المحلية (Drift / SQLite)

مشروع رزمة · ملف شرح للفريق

الغرض

تخزّن قاعدة البيانات على الجهاز كل ما يخص الطلاب: ملفاتهم وتقدّمهم ومحاولاتهم. لا شيء يُرسل للإنترنت الآن. الملفات: `lib/core/db/` .

المبدأ الأهم: فصل البيانات حسب الملف

الهاتف غالبًا مشترك بين الإخوة. لذلك كل جدول بيانات يحمل `profileId` ، وكل استعلام في `ActivityDao` مفلتر به. **لا يرى طالب تقدّم أخيه.** وعند حذف ملف تُحذف بياناته تلقائيًا (`onDelete: cascade` ، ويتطلب `PRAGMA foreign_keys = ON` الذي نفعّله في `beforeOpen`).
الاختباران `profile_repository_test` (الفصل في القاعدة) و `flows_test` («بيانات كل ملف منفصلة») يتحققان من ذلك.

الجداول

`profiles` — ملف الطالب

العمود	المعنى
`id`	معرّف uuid (مفتاح أساسي). لا نستخدم الاسم لأنه قد يتكرر أو يتغيّر.
`name`	اسم الطالب (1–40 حرفًا).
`grade`	الصف 1–12، مع `CHECK` في القاعدة.
`color`	لون الشخصية: teal / coral / blue / purple / amber (يُخزَّن اسمًا نصيًّا).
`pinSalt` , `pinHash`	الرمز السري **مجزّأً**. الاثنان `null` = لا رمز سري. انظر `02_profiles_and_pin.md` .
`studentCode`	رمز الطالب `RZ-XXXX` ، **فريد** (`UNIQUE`).
`classCode`	رمز الصف الاختياري من المعلم.
`createdAt` , `lastActiveAt`	للترتيب (الأحدث نشاطًا أولًا) وللملف النشط الأخير.

`lesson_progress` — تقدّم الدروس

مفتاح أساسي مركّب `(profileId, lessonId)` : صف واحد لكل طالب لكل درس. / `status` : `notStarted` / `downloaded` / `inProgress` / `completed` . و `progress` من 0 إلى 1. نستخدم `insertOnConflictUpdate` فتكون كتابة التقدّم «إنشاء أو تحديث» بأمر واحد. **كيف تستعمله الرئيسية:** أي صف حالته غير `notStarted` يُعدّ «نشاطًا» (فيخرج الطالب من حالة أول يوم)، وأحدث `inProgress` هو درس «المتابعة»، وعدد الـ `completed` ÷ دروس المادة = نسبة المادة.

`attempts` — المحاولات

كل محاولة تدريب (`practice`) أو اختبار (`quiz`): `activityId` , `score` , `total` , `starsEarned` , `createdAt` . هي مصدر **النجوم** (مجموع `starsEarned`) و**السلسلة** (أيام فيها محاولة) وقسم **المراجعة** (وجود `quiz`) وخطوتي «أكمل أول تدريب» و«جرّب أول اختبار».

`concept_mastery` — إتقان المفاهيم

مفتاح مركّب `(profileId, conceptId)` وقيمة `mastery` من 0 إلى 1. منها يُبنى «يحتاج إلى مراجعة»: المفاهيم ذات الإتقان **أقل من 0.6** (`kReviewMasteryThreshold`).

`sync_queue` — طابور المزامنة

`tries` , `createdAt` , `payloadJson` , `entity` . **يمتلئ الآن ولا يُرسل**. حين نبني المزامنة (Supabase) يقرأ منه ويرسل. وجوده الآن يعني ألا نفقد أي نتيجة عند ظهور الإنترنت.

`app_settings` — إعدادات الجهاز (جدول أضفناه)

مفتاح/قيمة. نستخدمه لحفظ **الملف النشط الأخير** (`active_profile_id`). أضفناه لأن هذه القيمة تخص الجهاز لا الطالب، وليست مكانًا مناسبًا في `profiles` .

قيم تُحسب ولا تُخزَّن

القيمة	تُحسب من
النجوم	مجموع `attempts.starsEarned` للملف
السلسلة	أيام نشاط متتالية حتى اليوم (`core/utils/streak.dart`)
نسبة تقدّم المادة	دروس `completed` ÷ دروس المادة في الكتالوج
«خطواتك الأولى X من 3»	وجود تقدّم/تدريب/اختبار

السبب: القيمة المخزّنة قد تخالف مصدرها (نجوم 10 في عمود ومحاولات تجمع 12). الحساب من المصدر لا يخطئ.

DAOs ودوالّها

DAO	الدالة	وظيفتها
`ProfilesDao`	`watchAll` / `getAll`	كل الملفات، الأحدث نشاطًا أولًا
	`watchById` / `getById`	ملف بمعرّفه (الثانية تتحدّث تلقائيًا)
	`getByStudentCode`	للتحقق من تفرّد رمز الطالب
	`count`	عدد الملفات
	`insertProfile`	إنشاء ملف
	`setPin` / `clearPin`	تعيين الرمز السري مجزّأً أو إلغاؤه
	`touch`	تحديث `lastActiveAt`
	`setClassCode`	تعديل رمز الصف
`ActivityDao`	`watchLessonProgress` / `upsertLessonProgress`	تقدّم الدروس
	`watchAttempts` / `insertAttempt`	المحاولات
	`watchMastery` / `upsertMastery`	إتقان المفاهيم

DAO	الدالة	وظيفتها
	`watchStarsByProfile`	**جديد:** مجموع نجوم كل ملف بـ `GROUP BY` (شاشة اختيار الملف)
	`clearActivity`	**جديد:** يمسح نشاط ملف واحد دون حذف الملف (لأدوات التطوير)
	`enqueueSync` / `watchPendingSyncCount`	طابور المزامنة وعدد المنتظر
`SettingsDao`	`read` / `write` / `remove`	مفتاح/قيمة (الملف النشط)

الـ DAO لا يحتوي قواعد عمل: القواعد في المستودعات.

`AppDatabase`

- `AppDatabase(executor)`: للاختبارات (تمرّر قاعدة في الذاكرة).
- `AppDatabase.onDevice()`: القاعدة الحقيقية عبر `driftDatabase(name: 'rizma')`، مع إعداد `web:` لملفي `sqlite3.wasm` و`drift_worker.js` عند التشغيل على الويب.
- `schemaVersion = 1`. عند تعديل جدول: أعد التوليد وارفع الرقم وأضف خطوة ترحيل (`onUpgrade`) قبل أي إصدار يُوزَّع.
- `beforeOpen` يفعّل المفاتيح الأجنبية.

التوليد التلقائي

Drift يولّد ملفات `*.g.dart` (فئات البيانات والاستعلامات) بالأمر:

```
dart run build_runner build
```

نبقي الملفات المولَّدة داخل Git.

الاختبار

`test/features/profile_repository_test.dart` يستخدم `AppDatabase(NativeDatabase.memory())`، قاعدة حقيقية في الذاكرة بلا ملفات. **تنبيه للاختبارات التي تكتب أثناء عرض واجهة:** لا تضع كتابتين متتاليتين داخل `tester.runAsync` واحد؛ استعلام الـ stream (في الزمن الوهمي) يمسك قفل القاعدة فتتعلّق الكتابة الثانية. اكتب كل عملية في استدعاء منفصل (انظر `12_testing.md`).